

Recursive Queries in PostgreSQL: Trees, Graphs, Org Charts, BOM

Table of Contents

- 1. [Learning Objectives](#)
 - 2. [Theory: Recursive Data Structures](#)
 - 3. [Recursive CTE Mechanics](#)
 - 4. [Org Chart Traversal](#)
 - 5. [Binary Tree Traversal](#)
 - 6. [Bill of Materials \(BOM\)](#)
 - 7. [Graph Traversal \(Shortest Path\)](#)
 - 8. [File System Simulation](#)
 - 9. [Generating Sequences](#)
 - 10. [Cycle Detection and Prevention](#)
 - 11. [ASCII Visual Diagrams](#)
 - 12. [Common Mistakes](#)
 - 13. [Best Practices](#)
 - 14. [Performance Considerations](#)
 - 15. [Interview Questions & Answers](#)
 - 16. [Hands-on Exercises](#)
 - 17. [Advanced Notes](#)
 - 18. [Cross-references](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Traverse any tree structure stored as adjacency list in a relational table
 - Implement Bill of Materials (BOM) explosion queries
 - Detect and prevent cycles in graph traversal
 - Compute path strings and depths for hierarchical data
 - Solve real-world recursive data problems in interviews
-

Theory: Recursive Data Structures

How Hierarchical Data is Stored (Adjacency List Model)

Each row has a reference to its parent row in the same table:

employees table:

emp_id	emp_name	manager_id (FK → emp_id)	
1	Alice	NULL	← root (no parent)
2	Bob	1	← child of Alice
3	Carol	1	← child of Alice
4	David	2	← child of Bob
5	Eve	2	← child of Bob

6 | Frank | 3 ← child of Carol

Tree:

```
Alice (1)
├── Bob (2)
│   ├── David (4)
│   └── Eve (5)
└── Carol (3)
    └── Frank (6)
```

Why Regular SQL Fails for Trees

A simple JOIN can only traverse one level. You'd need N JOINS for N levels, and the depth is unknown at query time. Recursive CTEs solve this by iteratively expanding the result.

Recursive CTE Mechanics

```
WITH RECURSIVE cte_name (col1, col2, ...) AS (
    -- ANCHOR MEMBER: seeds the recursion
    SELECT ...
    FROM table
    WHERE <base_condition>

    UNION ALL -- always UNION ALL for performance

    -- RECURSIVE MEMBER: extends by one level
    SELECT ...
    FROM table
    JOIN cte_name ON <join_condition>
    WHERE <termination_guard> -- optional depth limit
)
SELECT * FROM cte_name;
```

Schema Setup

```
-- Extended org hierarchy
CREATE TABLE org_employees (
    emp_id      SERIAL PRIMARY KEY,
    emp_name    VARCHAR(50) NOT NULL,
    title       VARCHAR(50),
    dept        VARCHAR(30),
    manager_id  INT REFERENCES org_employees(emp_id),
    salary      NUMERIC(10,2)
);

INSERT INTO org_employees (emp_name, title, dept, manager_id, salary) VALUES
('Alice CEO', 'CEO', 'Executive', NULL, 200000),
('Bob CTO', 'CTO', 'Engineering', 1, 150000),
('Carol CMO', 'CMO', 'Marketing', 1, 140000),
```

```

('David VP',      'VP Eng',      'Engineering', 2,    120000),
('Eve Dir',       'Dir Marketing','Marketing',   3,    100000),
('Frank Mgr',     'Eng Manager',  'Engineering', 4,    95000),
('Grace Dev',     'Senior Dev',   'Engineering', 6,    88000),
('Henry Dev',     'Dev',          'Engineering', 6,    75000),
('Iris Mkt',      'Mkt Specialist','Marketing',   5,    70000);

-- Bill of Materials
CREATE TABLE bom (
  component_id  INT PRIMARY KEY,
  component_name VARCHAR(50),
  parent_id     INT REFERENCES bom(component_id),
  quantity      NUMERIC(8,3),
  unit_cost     NUMERIC(10,2)
);

INSERT INTO bom VALUES
(1, 'Bicycle',      NULL, 1, NULL),
(2, 'Frame',        1,   1, 150.00),
(3, 'Wheel (Front)', 1,   1,  80.00),
(4, 'Wheel (Rear)',  1,   1,  80.00),
(5, 'Rim',          3,   1,  25.00),
(6, 'Rim',          4,   1,  25.00),
(7, 'Tire',         3,   1,  30.00),
(8, 'Tire',         4,   1,  30.00),
(9, 'Spoke',        5,  32,   0.50),
(10, 'Spoke',       6,  32,   0.50),
(11, 'Pedal Assy',  1,   2,  25.00),
(12, 'Pedal Body', 11,   1,   8.00),
(13, 'Pedal Axle', 11,   1,   5.00);

```

Org Chart Traversal

```

-- Example 1: Full org chart with indentation and depth
WITH RECURSIVE org_tree AS (
  -- Anchor: top-level (CEO)
  SELECT
    emp_id,
    emp_name,
    title,
    dept,
    manager_id,
    salary,
    0 AS depth,
    emp_name::TEXT AS path,
    ARRAY[emp_id] AS ancestors
  FROM org_employees
  WHERE manager_id IS NULL

  UNION ALL

```

```

-- Recursive: direct reports of each already-processed employee
SELECT
    e.emp_id,
    e.emp_name,
    e.title,
    e.dept,
    e.manager_id,
    e.salary,
    ot.depth + 1,
    ot.path || ' > ' || e.emp_name,
    ot.ancestors || e.emp_id
FROM org_employees e
JOIN org_tree ot ON e.manager_id = ot.emp_id
)
SELECT
    REPEAT(' ', depth) || emp_name AS hierarchy,
    title,
    dept,
    salary,
    depth,
    path
FROM org_tree
ORDER BY path;

-- Example 2: Find all subordinates of Bob CTO (emp_id = 2)
WITH RECURSIVE subordinates AS (
    SELECT emp_id, emp_name, title, dept, salary, 0 AS level
    FROM org_employees WHERE emp_id = 2

    UNION ALL

    SELECT e.emp_id, e.emp_name, e.title, e.dept, e.salary, s.level + 1
    FROM org_employees e
    JOIN subordinates s ON e.manager_id = s.emp_id
)
SELECT REPEAT(' ', level) || emp_name AS name, title, salary, level
FROM subordinates
ORDER BY level, emp_name;

-- Example 3: Find the reporting chain from Grace Dev up to CEO
WITH RECURSIVE upward_chain AS (
    SELECT emp_id, emp_name, title, manager_id, 0 AS level
    FROM org_employees WHERE emp_id = 7 -- Grace Dev

    UNION ALL

    SELECT e.emp_id, e.emp_name, e.title, e.manager_id, uc.level + 1
    FROM org_employees e
    JOIN upward_chain uc ON e.emp_id = uc.manager_id
)
SELECT REPEAT(' ', level) || emp_name AS chain, title, level

```

```

FROM upward_chain
ORDER BY level DESC;

-- Example 4: Org chart with total subordinate count
WITH RECURSIVE tree AS (
    SELECT emp_id, emp_name, manager_id, 0 AS depth
    FROM org_employees WHERE manager_id IS NULL

    UNION ALL

    SELECT e.emp_id, e.emp_name, e.manager_id, t.depth + 1
    FROM org_employees e JOIN tree t ON e.manager_id = t.emp_id
),
sub_counts AS (
    SELECT t1.emp_id, COUNT(t2.emp_id) AS subordinate_count
    FROM tree t1
    LEFT JOIN tree t2 ON t1.emp_id = ANY(
        (WITH RECURSIVE s(id) AS (
            SELECT emp_id FROM org_employees WHERE manager_id = t1.emp_id
            UNION ALL
            SELECT e.emp_id FROM org_employees e JOIN s ON e.manager_id = s.id
        ) SELECT id FROM s)
    )
    GROUP BY t1.emp_id
)
SELECT oe.emp_name, oe.title, sc.subordinate_count
FROM org_employees oe
JOIN sub_counts sc ON oe.emp_id = sc.emp_id
ORDER BY sc.subordinate_count DESC;

```

Bill of Materials (BOM)

```

-- Example 5: Full BOM explosion – all components of a Bicycle
WITH RECURSIVE bom_explosion AS (
    -- Anchor: top-level product
    SELECT
        b.component_id,
        b.component_name,
        b.parent_id,
        b.quantity,
        b.unit_cost,
        0 AS level,
        b.component_name::TEXT AS full_path,
        b.quantity AS total_quantity
    FROM bom b
    WHERE b.component_id = 1    -- Bicycle

    UNION ALL

    -- Recursive: sub-components

```

```

SELECT
    b.component_id,
    b.component_name,
    b.parent_id,
    b.quantity,
    b.unit_cost,
    be.level + 1,
    be.full_path || ' > ' || b.component_name,
    be.total_quantity * b.quantity -- propagate quantity through levels
FROM bom b
JOIN bom_explosion be ON b.parent_id = be.component_id
)
SELECT
    REPEAT(' ', level) || component_name AS component,
    total_quantity,
    unit_cost,
    ROUND(total_quantity * COALESCE(unit_cost, 0), 2) AS extended_cost,
    level,
    full_path
FROM bom_explosion
ORDER BY full_path;

-- Example 6: Total cost rollup per assembly
WITH RECURSIVE bom_cost AS (
    SELECT component_id, component_name, parent_id,
           COALESCE(unit_cost, 0) AS cost, quantity, 0 AS level
    FROM bom

    UNION ALL

    SELECT b.component_id, b.component_name, b.parent_id,
           bc.cost * b.quantity, b.quantity, bc.level + 1
    FROM bom b JOIN bom_cost bc ON b.component_id = bc.parent_id
)
SELECT component_name, SUM(cost) AS total_component_cost
FROM bom_cost
GROUP BY component_id, component_name
ORDER BY total_component_cost DESC;

```

Graph Traversal

```

-- Example 7: Graph – all paths from node A to all reachable nodes
CREATE TEMP TABLE graph_edges (
    from_node VARCHAR(5),
    to_node   VARCHAR(5),
    distance  INT
);

INSERT INTO graph_edges VALUES
    ('A', 'B', 4), ('A', 'C', 2),

```

```

('B', 'D', 5), ('B', 'C', 1),
('C', 'B', 1), ('C', 'D', 8), ('C', 'E', 10),
('D', 'E', 2), ('E', 'D', 3);

WITH RECURSIVE paths AS (
  -- Start from node A
  SELECT from_node, to_node, distance, ARRAY[from_node, to_node] AS visited
  FROM graph_edges
  WHERE from_node = 'A'

  UNION ALL

  -- Extend path by one edge
  SELECT p.from_node, e.to_node, p.distance + e.distance,
         p.visited || e.to_node
  FROM paths p
  JOIN graph_edges e ON p.to_node = e.from_node
  WHERE NOT e.to_node = ANY(p.visited) -- no cycles
         AND array_length(p.visited, 1) < 10 -- max depth guard
)
SELECT from_node, to_node, distance, visited AS path
FROM paths
ORDER BY to_node, distance;

-- Example 8: Shortest path (Dijkstra-like – simplified)
WITH RECURSIVE shortest AS (
  SELECT from_node AS start, to_node AS dest, distance, 1 AS hops,
         ARRAY[from_node, to_node] AS path
  FROM graph_edges WHERE from_node = 'A'

  UNION ALL

  SELECT s.start, e.to_node, s.distance + e.distance, s.hops + 1,
         s.path || e.to_node
  FROM shortest s
  JOIN graph_edges e ON s.dest = e.from_node
  WHERE NOT e.to_node = ANY(s.path) AND s.hops < 8
)
SELECT DISTINCT ON (dest)
       dest, distance, hops, path
FROM shortest
ORDER BY dest, distance;

```

File System Simulation

```

-- Example 9: File system tree
CREATE TEMP TABLE fs_nodes (
  node_id      INT PRIMARY KEY,
  name         VARCHAR(100),
  parent_id    INT,

```

```

    node_type    VARCHAR(10) -- 'dir' or 'file'
);

INSERT INTO fs_nodes VALUES
(1, '/',      NULL, 'dir'),
(2, 'home',   1,   'dir'),
(3, 'etc',    1,   'dir'),
(4, 'usr',    1,   'dir'),
(5, 'alice',  2,   'dir'),
(6, 'bob',    2,   'dir'),
(7, 'passwd', 3,   'file'),
(8, 'hosts',  3,   'file'),
(9, 'docs',   5,   'dir'),
(10, 'resume.pdf', 9, 'file'),
(11, 'notes.txt', 5, 'file');

WITH RECURSIVE file_tree AS (
    SELECT node_id, name, parent_id, node_type, 0 AS depth,
           '/' || name AS full_path
    FROM fs_nodes WHERE parent_id IS NULL

    UNION ALL

    SELECT n.node_id, n.name, n.parent_id, n.node_type, ft.depth + 1,
           ft.full_path || '/' || n.name
    FROM fs_nodes n
    JOIN file_tree ft ON n.parent_id = ft.node_id
)
SELECT
    REPEAT(' ', depth) || name AS file_tree_view,
    node_type,
    full_path,
    depth
FROM file_tree
ORDER BY full_path;

```

Generating Sequences

```

-- Example 10: Generate a sequence of numbers (alternative to generate_series)
WITH RECURSIVE nums(n) AS (
    SELECT 1
    UNION ALL
    SELECT n + 1 FROM nums WHERE n < 20
)
SELECT n FROM nums;

-- Example 11: Calendar for a month
WITH RECURSIVE cal(d) AS (
    SELECT DATE '2024-03-01'
    UNION ALL

```

```

        SELECT d + 1 FROM cal WHERE d < DATE '2024-03-31'
    )
SELECT
    d AS date,
    TO_CHAR(d, 'Day') AS day_of_week,
    EXTRACT(WEEK FROM d) AS week_num
FROM cal;

-- Example 12: Business day generator (skip weekends)
WITH RECURSIVE biz_days(d) AS (
    SELECT DATE '2024-01-01'
    UNION ALL
    SELECT
        CASE
            WHEN EXTRACT(DOW FROM d + 1) IN (0, 6)
            THEN d + (
                CASE EXTRACT(DOW FROM d + 1)
                    WHEN 6 THEN 2 -- Saturday → skip to Monday
                    WHEN 0 THEN 1 -- Sunday → skip to Monday
                END
            )
            ELSE d + 1
        END
    FROM biz_days
    WHERE d < DATE '2024-01-31'
)
SELECT d, TO_CHAR(d, 'Day') AS day FROM biz_days;

```

Cycle Detection and Prevention

```

-- Example 13: Detect cycles using visited array
WITH RECURSIVE safe_traverse AS (
    SELECT emp_id, emp_name, manager_id,
        ARRAY[emp_id] AS visited,
        FALSE AS cycle
    FROM org_employees
    WHERE manager_id IS NULL

    UNION ALL

    SELECT e.emp_id, e.emp_name, e.manager_id,
        st.visited || e.emp_id,
        e.emp_id = ANY(st.visited)
    FROM org_employees e
    JOIN safe_traverse st ON e.manager_id = st.emp_id
    WHERE NOT st.cycle
)
SELECT emp_name, cycle, visited
FROM safe_traverse
WHERE cycle = TRUE; -- shows any employee in a cycle

```

```
-- Example 14: PostgreSQL 14+ CYCLE clause
-- (if available – check your version)
WITH RECURSIVE org AS (
    SELECT emp_id, emp_name, manager_id FROM org_employees
    WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.emp_name, e.manager_id
    FROM org_employees e JOIN org ON e.manager_id = org.emp_id
)
CYCLE emp_id SET is_cycle USING cycle_path
SELECT emp_name, is_cycle, cycle_path
FROM org
WHERE is_cycle;
```

ASCII Visual Diagrams

Recursive CTE Execution Steps

Org chart recursion (Anchor = CEO):

Iteration 0 (anchor):

Working table: {Alice (id=1, depth=0)}

Iteration 1:

Join employees with working table on emp.manager_id = wt.emp_id

New rows: {Bob(2,1), Carol(3,1)}

Working table: {Bob, Carol}

Iteration 2:

Join again: {David(4,2), Eve(5,2) from Bob; Frank(6,3) from Carol... etc}

...

Final UNION ALL:

{Alice} ∪ {Bob, Carol} ∪ {David, Eve, Frank Mgr} ∪ {Grace, Henry, Iris}

BOM Tree

Bicycle (1)

```
|— Frame (2) – qty=1, cost=150
|— Wheel Front (3) – qty=1, cost=80
|   |— Rim (5) – qty=1, cost=25
|   |   |— Spoke (9) – qty=32, cost=0.50 each
|   |   |— Tire (7) – qty=1, cost=30
|— Wheel Rear (4) – qty=1, cost=80
|   |— Rim (6) – qty=1, cost=25
|   |   |— Spoke (10) – qty=32, cost=0.50 each
|   |   |— Tire (8) – qty=1, cost=30
|— Pedal Assy (11) – qty=2, cost=25
```

```
└─ Pedal Body (12) – qty=1, cost=8
└─ Pedal Axle (13) – qty=1, cost=5
```

Common Mistakes

Mistake 1: Forgetting termination condition

```
-- WRONG: recurses forever if no WHERE
WITH RECURSIVE r AS (SELECT 1 AS n UNION ALL SELECT n+1 FROM r)
SELECT * FROM r;

-- CORRECT:
WITH RECURSIVE r AS (SELECT 1 AS n UNION ALL SELECT n+1 FROM r WHERE n < 100)
SELECT * FROM r;
```

Mistake 2: Using UNION instead of UNION ALL

```
-- UNION deduplicates every iteration – O(N^2) cost, wrong for most uses
-- UNION ALL is always correct for tree traversal
```

Mistake 3: Not guarding against cycles in graph data

```
-- Without cycle detection on graph data:
-- infinite recursion, memory exhaustion
-- Always maintain a visited array:
WHERE NOT new_node = ANY(visited_path_array)
```

Mistake 4: Traversing upward without path tracking

```
-- Upward traversal (from leaf to root) also needs a cycle guard
-- even in well-structured org data, data entry errors can create loops
```

Best Practices

1. Always set a maximum depth guard (`WHERE depth < N`).
 2. Always use UNION ALL in recursive CTEs.
 3. Use a path array for cycle detection in graph data.
 4. Use SEARCH DEPTH FIRST / BREADTH FIRST (PG14+) to control traversal order.
 5. Use CYCLE clause (PG14+) for automated cycle detection.
 6. Index the parent_id / manager_id column heavily — it is used in every iteration.
 7. For very deep trees (100+ levels), increase `max_recursion_depth` .
-

Performance Considerations

```
-- 1. Index parent_id – each recursive step joins on this:
CREATE INDEX idx_org_manager ON org_employees(manager_id);
CREATE INDEX idx_bom_parent ON bom(parent_id);

-- 2. The recursive member executes once per level
-- For a 6-level org with 1000 employees, it runs ~6 times
-- With index, each run is fast (index lookup)

-- 3. Without index – each run does a full seq scan
-- Catastrophic for large tables

-- 4. Check max_recursion_depth and increase if needed:
SHOW max_recursion_depth; -- default 100 in PostgreSQL
SET max_recursion_depth = 500;

-- 5. Use MATERIALIZED CTE if the result is used multiple times:
WITH RECURSIVE org AS MATERIALIZED (...)
SELECT * FROM org o1 JOIN org o2 ON ...;
```

Interview Questions & Answers

Q1: What is the adjacency list model for hierarchical data?

A: Each row stores its own ID and a reference (foreign key) to its parent row in the same table. Leaf nodes have no children. Root nodes have no parent (NULL parent_id). This is the most common way to store trees in relational databases.

Q2: How does a recursive CTE work step by step?

A: The anchor member executes first and produces the initial result set (level 0). Then the recursive member joins the result table to the base table to produce the next level. This repeats until the recursive member produces no new rows. The final result is UNION ALL of all iterations.

Q3: What prevents infinite recursion in a recursive CTE?

A: Either a WHERE condition that limits depth (WHERE depth < 10), a cycle detection check (WHERE NOT node = ANY(visited)), or the PostgreSQL max_recursion_depth setting (default 100).

Q4: How do you perform a BOM explosion?

A: Use a recursive CTE starting from the top-level product (anchor), then join to components whose parent_id matches the current level's component_id. Multiply quantities at each level to get total quantities.

Q5: How do you traverse a graph (not a tree) without infinite loops?

A: Maintain an array of visited nodes as a CTE column. In each recursive step, check that the new node is NOT in the visited array: WHERE NOT new_node = ANY(visited) . PostgreSQL 14+ offers the native CYCLE clause for this.

Q6: What is depth-first vs breadth-first traversal in trees?

A: Depth-first traversal follows one branch all the way to the leaf before backtracking (uses a stack). Breadth-first visits all nodes at each level before going deeper (uses a queue). PostgreSQL 14+ supports SEARCH

DEPTH FIRST and SEARCH BREADTH FIRST clauses.

Q7: How would you find all employees at depth 2 in an org chart?

A: Add a depth counter in the recursive CTE (depth + 1 in the recursive member), then filter WHERE depth = 2.

Q8: Can you do a recursive CTE on two tables at once?

A: Yes, by joining the recursive CTE result to any number of tables in the recursive member. The recursive CTE only needs to reference itself once.

Hands-on Exercises

Exercise 1: Find Total Headcount Under Each Manager

For each employee in the org_employees table, count how many subordinates (direct and indirect) they manage.

```
-- Solution:
WITH RECURSIVE all_subs AS (
    SELECT emp_id AS root, emp_id AS sub_id FROM org_employees

    UNION ALL

    SELECT a.root, e.emp_id
    FROM all_subs a
    JOIN org_employees e ON e.manager_id = a.sub_id
)
SELECT
    oe.emp_name,
    oe.title,
    COUNT(a.sub_id) - 1 AS subordinate_count -- subtract self
FROM all_subs a
JOIN org_employees oe ON a.root = oe.emp_id
GROUP BY oe.emp_id, oe.emp_name, oe.title
ORDER BY subordinate_count DESC;
```

Exercise 2: BOM Quantity Rollup

For each component in the Bicycle BOM, calculate the total quantity needed.

```
-- Solution:
WITH RECURSIVE bom_qty AS (
    SELECT component_id, component_name, parent_id,
           1::NUMERIC AS total_qty, 0 AS depth
    FROM bom WHERE component_id = 1

    UNION ALL

    SELECT b.component_id, b.component_name, b.parent_id,
           bq.total_qty * b.quantity, bq.depth + 1
    FROM bom b
    JOIN bom_qty bq ON bq.parent_id = b.parent_id
)
```

```

        JOIN bom_qty bq ON b.parent_id = bq.component_id
    )
    SELECT
        component_name,
        SUM(total_qty) AS total_quantity,
        MAX(depth) AS max_depth
    FROM bom_qty
    GROUP BY component_id, component_name
    ORDER BY max_depth, component_name;

```

Exercise 3: Generate Working Days

Generate all business days (Mon-Fri) for February 2024.

```

-- Solution:
WITH RECURSIVE bdays(d) AS (
    SELECT DATE '2024-02-01'
    UNION ALL
    SELECT d + CASE
        WHEN EXTRACT(DOW FROM d) = 5 THEN 3 -- Friday → Monday
        ELSE 1
    END
    FROM bdays
    WHERE d < DATE '2024-02-29'
)
SELECT d, TO_CHAR(d, 'Day, DD Mon YYYY') AS formatted
FROM bdays
WHERE EXTRACT(DOW FROM d) BETWEEN 1 AND 5
ORDER BY d;

```

Exercise 4: Org Depth Report

For each department, find the maximum hierarchy depth (number of levels from CEO to deepest employee).

```

-- Solution:
WITH RECURSIVE depths AS (
    SELECT emp_id, dept, 0 AS depth
    FROM org_employees WHERE manager_id IS NULL

    UNION ALL

    SELECT e.emp_id, e.dept, d.depth + 1
    FROM org_employees e
    JOIN depths d ON e.manager_id = d.emp_id
)
SELECT dept, MAX(depth) AS max_levels
FROM depths
GROUP BY dept
ORDER BY max_levels DESC;

```

Advanced Notes

Closure Table Model (Alternative to Adjacency List)

For very frequent hierarchical queries, a closure table stores all ancestor-descendant pairs:

```
-- Pre-computed closure table
CREATE TABLE org_closure (
  ancestor_id INT,
  descendant_id INT,
  depth INT,
  PRIMARY KEY (ancestor_id, descendant_id)
);
-- All queries become simple JOIN + WHERE (no recursion needed)
-- Trade-off: expensive to maintain on INSERTs/DELETEs/MOVEs
```

ltree Extension (PostgreSQL)

For very large hierarchies, the `ltree` extension stores path strings and provides path operators:

```
CREATE EXTENSION ltree;
-- path: '1.2.6.7' represents Alice > Bob > Frank > Grace
-- Query all descendants: WHERE path <@ '1.2'
-- This is much faster than recursive CTEs for read-heavy workloads
```

Cross-references

- [04_ctes.md](#) — CTE fundamentals and materialization
- [01_joins_complete.md](#) (03_Intermediate_SQL) — SELF JOIN as a one-level alternative
- [08_interview_window_tasks.md](#) — Recursive patterns appear in company problems
- [10_PostgreSQL_Internals/](#) — ltree extension for production hierarchies